

UADE

Ingeniería de Datos II

Trabajo Práctico Obligatorio 2026

Evangelina Ovelar - Santino Lozano

Introducción

El presente trabajo propone una solución de persistencia políglota para un sistema de facturación. El objetivo es registrar clientes, teléfonos, productos, facturas y detalles de factura, contemplando la disponibilidad de stock, la actualización de cantidades vendidas y el cálculo de importes con IVA y descuentos por volumen.

La propuesta utiliza MySQL como base transaccional principal y MongoDB como base documental complementaria. Esta combinación permite cubrir tanto la integridad de las operaciones de venta como las consultas rápidas orientadas a reportes.

Alcance del sistema

El sistema permite administrar clientes y productos, registrar facturas, consultar la relacion entre clientes, teléfonos y ventas, identificar productos facturados y no facturados, y obtener información agregada sobre el gasto total de cada cliente.

Req.	Descripcion
1	Obtener datos de clientes junto con sus teléfonos.
2	Obtener teléfonos y número de cliente de Jacob Cooper.
3	Mostrar cada teléfono junto con los datos del cliente.
4	Obtener clientes con al menos una factura.
5	Identificar clientes sin facturas.
6	Devolver clientes con cantidad de facturas, considerando 0 cuando no tengan.
7	Listar facturas compradas por Kai Bullock.
8	Seleccionar productos facturados al menos una vez.
9	Listar facturas que contengan productos de la marca Ipsum.
10	Mostrar nombre y apellido de cada cliente junto con el gasto total con IVA incluido.
11	Crear una vista de facturas ordenadas por fecha.
12	Crear una vista de productos no facturados.
13	Crear, eliminar y modificar clientes.
14	Crear y modificar productos, considerando que el precio es sin IVA.

Diseño de persistencia políglota

La capa de persistencia se divide en dos bases de datos, cada una con una responsabilidad clara.

Base	Rol	Justificacion
MySQL	Base principal transaccional	Guarda clientes, teléfonos, productos, facturas y detalles. Controla claves foráneas, stock, IVA, descuentos y operaciones atómicas.
MongoDB	Base documental complementaria	Guarda facturas desnormalizadas con cliente e items embebidos para consultas de lectura y reportes.

Flujo propuesto:

- El sistema registra la operación en MySQL, donde se asegura la consistencia de la venta. Luego, para consultas y reportes, se puede replicar una versión documental de la factura en MongoDB. De esta forma, MySQL queda como fuente de verdad y MongoDB como modelo optimizado para lectura.

Modelo relacional en MySQL

MySQL se utiliza para el núcleo del sistema porque las facturas, los productos y el stock requieren consistencia. Una venta no debe registrarse parcialmente: si no hay stock suficiente para algún item, la factura completa debe cancelarse.

Tabla	Clave primaria	Responsabilidad
E01_CLIENTE	nro_cliente	Datos básicos del cliente. El campo activo permite baja lógica.
E01_TELEFONO	nro_cliente, cod_area, nro_telefono	Teléfonos asociados a clientes. La clave compuesta evita teléfonos duplicados para un mismo cliente.
E01_PRODUCTO	codigo_producto	Productos con marca, nombre, precio sin IVA y stock.
E01_FACTURA	nro_factura	Cabecera de factura con fecha, totales y cliente asociado.
E01_DETALLE_FACTURA	nro_factura, nro_item	Ítems facturados. Se agregan precio histórico, descuento y subtotal para conservar la información de la venta.

Justificación de decisiones de diseño

Decisión	Justificación
Uso de MySQL	Se eligió para la parte transaccional porque permite claves primarias, claves foráneas, restricciones y transacciones ACID.
Uso de MongoDB	Se eligió para consultas y reportes porque permite guardar facturas como documentos completos, evitando joins frecuentes en lecturas.
DECIMAL para importes	Los importes monetarios se definieron como DECIMAL para evitar errores de redondeo propios de FLOAT o DOUBLE.
Baja logica de clientes	El campo activo permite deshabilitar clientes sin perder el historial de facturación.
Restricciones referenciales	Las facturas y detalles referencian clientes y productos existentes, evitando datos huérfanos.
Detalle con precio histórico	Aunque el DER original contiene el producto, se agregó precio unitario sin iva para conservar el valor real facturado si el precio cambia luego.
IVA centralizado	El IVA se calcula al 21% dentro del procedimiento de facturación para mantener un criterio único.
Descuento por volumen	Se adoptó una regla simple: 0% hasta 4 unidades, 5% desde 5 unidades y 10% desde 10 unidades.
Control de stock atomico	La actualización de stock se realiza dentro de una transacción. Si algún producto no alcanza, se revierte toda la operación.
Vistas	Las vistas resuelven consultas reutilizables: facturas ordenadas y productos no facturados.

Logica de facturación

La operación principal se implementa mediante el procedimiento `sp_crear_factura`. Este procedimiento recibe el cliente y los productos a facturar, valida que el cliente exista y esté activo, verifica que existan los productos, controla stock disponible, descuenta las unidades vendidas, calcula subtotales, aplica descuentos por volumen, calcula IVA y actualiza la factura con los totales finales.

Paso	Descripcion
1	Validar que la factura tenga items.
2	Validar que el cliente exista y esté activo.
3	Agrupar productos recibidos para evitar duplicados dentro de la misma factura.
4	Comprobar la existencia de productos.
5	Actualizar stock de forma atomica.
6	Insertar cabecera de factura.
7	Insertar detalle con precio histórico, descuento y subtotal.
8	Calcular total sin IVA, IVA y total con IVA.
9	Confirmar la transacción o revertirla ante cualquier error.

Ejemplo de llamada al procedimiento:

```
CALL sp_crear_factura(2, JSON_ARRAY(  
  JSON_OBJECT('codigo_producto', 1, 'cantidad', 2),  
  JSON_OBJECT('codigo_producto', 3, 'cantidad', 1)  
));
```

Modelo documental en MongoDB

MongoDB se usa como repositorio complementario de lectura. En lugar de separar factura, cliente, producto y detalle en varias tablas, cada factura se guarda como un documento que contiene una foto del cliente y de los productos facturados.

Coleccion	Contenido
clientes	Datos del cliente con teléfonos embebidos en un arreglo.
productos	Catálogo de productos con precio sin IVA y stock informativo.
facturas	Documento de factura con cliente embebido, items embebidos, subtotal, IVA y total.

La ventaja de este modelo es que las consultas de reportes, como facturas por cliente, productos facturados y totales por cliente, pueden resolverse leyendo documentos completos. La desnormalización es intencional y se justifica porque MongoDB no se usa como fuente transaccional principal, sino como apoyo para lectura.

Mapeo de requerimientos a implementación

Req.	Implementacion
1	JOIN entre E01_CLIENTE y E01_TELEFONO / consulta a clientes.telefonos.
2	Filtro por nombre y apellido Jacob Cooper.
3	LEFT JOIN desde telefonos o \$unwind del arreglo telefonos.
4	INNER JOIN con E01_FACTURA o agrupacion de facturas por cliente.
5	LEFT JOIN con filtro IS NULL o \$lookup con arreglo vacio.
6	COUNT de facturas por cliente o \$lookup + \$size.
7	Filtro de facturas por cliente Kai Bullock.
8	DISTINCT de productos presentes en detalle o \$unwind de items.
9	JOIN con productos por marca Ipsum o busqueda en items.marca.
10	SUM de total_con_iva por cliente.
11	Vista V_FACTURAS_ORDENADAS / view v_facturas_ordenadas.
12	Vista V_PRODUCTOS_NO_FACTURADOS / view v_productos_no_facturados.
13	Procedimientos de cliente y funciones CRUD de apoyo.
14	Procedimientos de producto y funciones CRUD de apoyo.

Archivos entregados

Archivo	Contenido
01_mysql_sistema_facturacion.sql	Script completo de MySQL con creacion de base, tablas, indices, procedimientos, datos de prueba, vistas y consultas.
02_mongodb_sistema_facturacion.js	Script completo de MongoDB con colecciones, validaciones, documentos, indices, vistas, consultas y funciones CRUD.
README_EJECUCION_TPO.txt	Instrucciones breves para ejecutar los scripts.

Conclusión

La solución propuesta cumple con los requerimientos del sistema y aplica persistencia polígloa de manera justificada. MySQL resuelve la integridad y la consistencia necesarias para facturación y stock, mientras que MongoDB aporta un modelo documental adecuado para consultas y reportes. La separación de responsabilidades evita forzar una sola tecnología para todos los casos de uso y permite una implementación más clara y mantenible.